

Syntax Analyzer for IF-ELSE Statements using Flex and Bison

Aim

- To design a syntax analyzer that accepts valid `if`, `if-else`, and nested `if-else` statements.
- To detect syntax errors in the provided statements.
- To resolve the dangling-else problem using grammar precedence and associativity rules.
- To parse statements interactively via the command line interface.

Software Requirements

Tool	Purpose
Flex	Lexical Analyzer Generator (Tokenizes the input stream)
Bison	Parser Generator (Validates syntax using Context-Free Grammar)
GCC	C Compiler to compile the generated source files
OS	Windows (MinGW environment) / Linux

Theory

- **Flex Component:** Scans the input character stream and converts it into a sequence of tokens such as `IF`, `ELSE`, `ID`, `RELOP`, and `NUM`.
- **Bison Component:** Utilizes a Context-Free Grammar (CFG) to parse the semantic tokens produced by Flex and structurally validate the execution block.
- **Dangling Else Resolution:** The inherent ambiguity of the dangling else problem is gracefully resolved within Bison by declaring token precedence rules (`%nonassoc`) ensuring that an `else` aligns with the nearest open `if`.

Program Structure

1. Lexical Analyzer File: ifelse.l

```
%{
#include "ifelse.tab.h"
#include <stdlib.h>
%}
%%
"if"          { return IF; }
"else"        { return ELSE; }
"="           { return ASSIGN; }
";"           { return SEMICOLON; }
"("           { return '('; }
")"           { return ')'; }
[<>!=]=?      { return RELOP; }
[0-9]+        { yylval = atoi(yytext); return NUM; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
[\\t \\r]     { /* skip whitespace */ }
[\\n]         { return 0; /* signal end-of-input for this
yyparse() call */ }
.             { printf("Unknown symbol: %s\\n", yytext); }
%%
int yywrap() {
    return 1;
}
```

2. Parser File: ifelse.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
void yyerror(const char *s);
int isError = 0;
%}
%token IF ELSE ID RELOP ASSIGN NUM SEMICOLON
%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%start stmt
%%
stmt : if_stmt {
        if (!isError)
            printf("valid\\n");
    }
    ;
```

```

if_stmt : IF '(' cond ')' stmt_block %prec LOWER_THAN_ELSE
        | IF '(' cond ')' stmt_block ELSE stmt_block
        ;
stmt_block : ID ASSIGN expr SEMICOLON
           | if_stmt
           ;
cond : ID RELOP ID
     | ID RELOP NUM
     ;
expr : ID
     | NUM
     ;
%%
void yyerror(const char *s) {
    isError = 1;
    printf("Syntax Error: Invalid statement\n");

    // Flush remaining tokens on the current line using the lexer itself,
    // not raw stdio, so we stay in sync with flex's internal buffer.
    while (yylex() != 0)
        ;
}

int main() {
    printf("=== IF-ELSE Statement Parser ===\n");
    printf("Enter one statement per line, eg:\n");
    printf("if (a>b) x=y;\n");
    printf("if (a>b) x=y; else x=z;\n");
    printf("Press Ctrl+C or Ctrl+Z (Windows) to exit.\n\n");
    while (1) {
        isError = 0;
        printf("> ");
        if (yyparse() != 0 && feof(stdin))
            break;
    }
    return 0;
}

```

Compilation and Execution Steps

Execute the following commands sequentially in your terminal/command prompt to build and run the parser:

```

$ bison -d ifelse.y
$ flex ifelse.l
$ gcc ifelse.tab.c lex.yy.c -o ifelsechecker
$ ./ifelsechecker

```